Detection of intrusions and attacks in computer networks

Szymon Kluska, Kacper Krzesiński

***Abstract*—The aim of the project is to develop, implement, train and evaluate a model that will be used to detect intrusions and attacks in a computer network. An Autoencoder model, trained using the *open-set recognition*. The project will also include analysis and compilation of the results obtained.**

***Index Terms*—computer networks, network attacks, machine learning, Autoencoder, open-set recognition**

I. Wstęp

NETWORK infrastructure is constantly exposed to various types of threats. Analysis of data collected after incidents enables specialists to identify intrusion methods and, consequently, to create effective defence mechanisms. However, attackers are constantly developing their techniques and introducing new forms of abuse to remain undetectable. For this reason, machine learning-based systems must continuously monitor numerous telemetry indicators in real time to detect even minor deviations and generate rapid alerts. Equally important, however, is the ability to recognise completely new, previously unseen threats. It is crucial for network security that the early warning system responds even when it encounters a given attack for the first time.

II. Description of the concept being developed

The task is to develop, implement, train and evaluate a model that will be used to detect intrusions and attacks in a computer network. This model will be trained using the method *open-set recognition* [11], on data labelled as 'no attack'. The goal will be to teach the model to correctly assign normal network traffic to the 'no attack' class. However, if a sample does not fit into this class, the model will detect it as an anomaly, which should be interpreted as a potential attack. A model that works in this way, i.e. matching data to only one class and detecting anomalies, is called *Autoencoder* [10].

Training, validation and testing of the model will be carried out on publicly available data available on the Internet [1, 2] and will be converted into a binary label - attack vs no attack.

The aim of the project will be to answer the following questions (*research questions*):

Q1) Do models of the *Autoencoder* type, trained using the *open-set recognition* method, correctly classify "normal" network traffic?

Q2) Do models of the *Autoencoder* type, trained using the *open-set recognition* method, effectively detect both commonly known and novel network attacks?

To answer these questions, a model of the *Autoencoder* type will be implemented, and experiments will subsequently be conducted.

The quality of the predictions will be evaluated using the F1-score described by Equation 1.

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{1}$$

Where:

- TP (*True Positive*) – correctly detected attack.
- FP (*False Positive*) – false alarm.
- FN (*False Negative*) – missed attack.
- TN (*True Negative*) – correctly recognized normal traffic.

The model will be implemented using the Python programming language [3] together with the *scikit-learn* [4] and *PyTorch* [5] libraries. The *pandas* [6], *NumPy* [7], and *matplotlib* [8] libraries will be used for data processing and representation.

III. Algorithms

An LSTM Autoencoder is a neural network composed of an encoder–decoder pair. The encoder reads the input sequence and transforms the input data, while the decoder is responsible for reconstructing the original sequence. In the case of normal network traffic, the reconstruction error will be negligible, whereas in the case of anomalies, it will be significant [10].

A key issue is determining the reconstruction error threshold (*threshold*) above which a given sample will be considered an attack. This threshold should be calculated using the training data, i.e., purely normal traffic. To achieve this, all training packets must be passed through the trained model and the reconstruction error must be computed for each packet. The errors are then sorted in ascending order, and a decision about the threshold value is made. For example, if the 90th percentile is selected, the algorithm identifies the point below which 90% of the smallest errors fall. The value at this point becomes the reconstruction error threshold (*threshold*).

When a new packet from the test set is fed into the network, the model processes it and computes the reconstruction error. If this error exceeds the established threshold, the model raises an alarm; otherwise, it classifies the traffic as normal.

Network data are characterized by the presence of extreme outliers, e.g., massive file transfers versus small ping packets.

For this reason, the *RobustScaler* [13] algorithm was used for data scaling and preprocessing. It is a normalization method that uses the median and interquartile range instead of the mean and standard deviation, which helps preserve the dynamics of "typical" network traffic. Additionally, a logarithmic transformation was applied to reduce the order of magnitude of the data, facilitating neural network convergence and preventing volumetric features from dominating other attributes.

When working with *RobustScaler*, which generates negative values for half of the data, it is crucial to use the *LeakyReLU* activation function [14]. It is a variant of the *ReLU* function that does not completely zero out negative inputs, but instead allows them to pass through with a small multiplier (e.g., 0.1). *LeakyReLU* prevents the problem of "dead neurons" and enables the network to learn differences also in packets with values below the median.

The applied loss function is *MSELoss* [15]. It is an objective function that computes the mean of the squared differences between the input values and those reconstructed by the *Autoencoder*. By squaring the differences, this function penalizes large errors disproportionately while remaining tolerant to small deviations. In the context of cybersecurity, this means that an attack packet that drastically deviates from the norm generates a very high error value, which facilitates its separation from the background of normal network traffic.

The experiments were conducted using the k-fold cross-validation procedure (*k-fold cross-validation*). This method involves randomly dividing the entire available dataset into k disjoint subsets (so-called *folds*) of equal size. The evaluation process is repeated k times: in each iteration, one subset serves as the test set, while the remaining $k-1$ subsets form the training set. The use of this approach was motivated by the need to obtain a reliable and stable assessment of the model's performance, independent of a specific split of the data into training and validation parts. Cross-validation eliminates this problem by ensuring that each sample in the dataset is used exactly once for testing and $k-1$ times for training, and the final result is the arithmetic mean of all runs. In this study, the parameter k was set to 5, and in each iteration the model was trained for 5 epochs [17].

The implemented *Autoencoder* was compared with the *Isolation Forest* and *OC SVM* models on the KDD99 dataset.

Isolation Forest is an unsupervised learning algorithm based on the assumption that anomalies are rare events that significantly differ from the rest of the population. The algorithm constructs an ensemble of random binary trees, in which the data are recursively partitioned using randomly selected attributes and split points. In the resulting structure, attacks are separated from the rest of the dataset very early, ending up at shallower levels of the tree, whereas dense clusters of normal traffic require a much larger number of splits. The final anomaly score is therefore calculated based on the average path length required to isolate a given point – the shorter this path, the higher the probability that the examined sample constitutes a threat [14].

One-Class Support Vector Machine is an unsupervised learning algorithm used for novelty detection, which modifies the classical SVM approach by abandoning the search for a boundary between two classes in favor of determining a tight envelope around data representing normal traffic. This mechanism uses the so-called *kernel trick* to project the input data into a high-dimensional feature space, where the algorithm attempts to find an optimal hyperplane separating the cluster of normal data from the origin with the maximum possible margin. As a result, a nonlinear decision boundary is created that defines the region of "normality". Any new observations that, after transformation, lie outside this boundary (i.e., closer to the origin) are classified as anomalies [15]. In this study, due to the high computational complexity of the classical implementation (on the order of $O(n^3)$) and the large volume of network data, an optimized variant using Stochastic Gradient Descent (SGD) was applied. This reduced the complexity to linear time $O(n)$, enabling efficient processing of datasets containing millions of samples [16].

IV. Dataset Analysis

The model evaluation was conducted using three diverse datasets representing different characteristics of network traffic and feature formats. The first one is the *KDD Cup 1999 Data* [12], commonly used to compare the effectiveness of IDS systems. The second data source was a dataset based on the *NetFlow v9* [13] format. The third dataset, *CoReS-IoT*, focuses on the specifics of the Internet of Things and is distinguished by its labeling structure – it provides only binary classification (normal/attack) in the last column. This selection of input data enabled a comprehensive verification of the model's generalization capability and a thorough assessment of its effectiveness.
Dataset sizes:

- *KDD Cup 1999 Data* – 1,074,992 samples. Number of features: 39
- *NetFlow V9* – 1,000,000 samples. Number of features: 19
- *CoReS-IoT* – 1,008,748 samples. Number of features: 19

A. KDD Dataset

Figure 1 presents the transformation of the distribution of the *src_bytes* feature in the KDD dataset, which represents the number of source bytes. On the left side, we observe an "irregular" distribution, where the data do not have a clear center. For a neural network, such a distribution is difficult to interpret because the weights would need to adapt to a very wide range of values, which may lead to training instability. The application of *RobustScaler*, which uses the median resistant to extreme values for scaling, transformed the highly skewed distribution of network data into a distribution closer to normal and centered around zero. This is crucial for

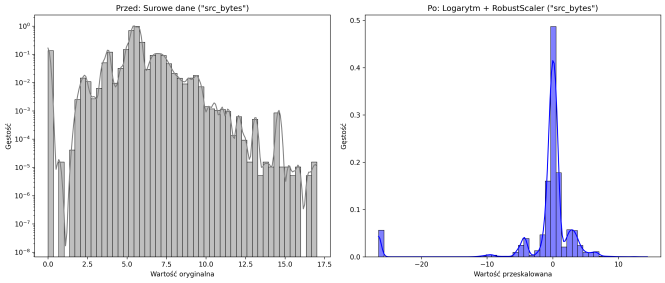

Fig. 1. Values of the *src_bytes* feature before and after scaling

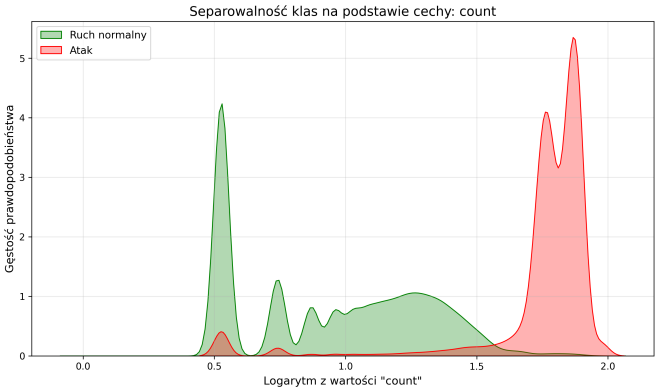

Fig. 2. Class separability based on the *count* feature

the stability of deep neural network training and for preventing the model from being dominated by extreme values.

The plot in Figure 2 presents a probability density analysis for the *count* feature after logarithmic transformation. This feature represents the number of connections to the same destination host within the last 2 seconds. The plot shows a very clear separation between normal traffic and attacks, indicating that the *count* feature may be a key predictor in the anomaly detection process.

B. NetFlow Dataset

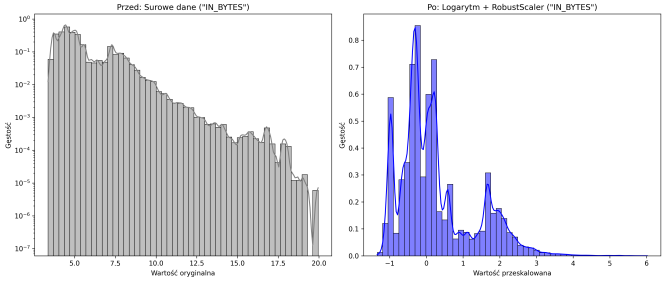

Fig. 3. Values of the *IN_BYTES* feature before and after scaling

Figure 3 presents the transformation of the distribution of the *in_bytes* feature in the NetFlow dataset, which, similarly to the KDD dataset, represents the number of source bytes. Again, it can be observed that the data on the left do not form a normal distribution and exhibit multiple local peaks. The application of *RobustScaler* highlighted the existence of different traffic classes (visible as separated "humps" in the right plot), which were blurred in the left plot due to scale differences. Bringing the data into a narrow range around zero enabled stable training and prevented the model from being dominated by extreme values.

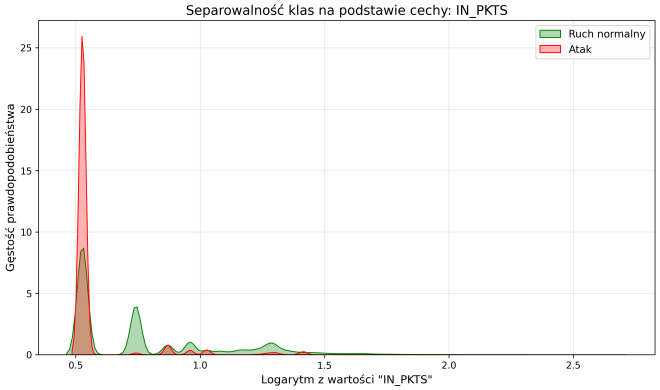

Fig. 4. Class separability based on the *IN_PKTS* feature

The plot in Figure 4 presents a probability density analysis for the *in_pkts* feature after logarithmic transformation. We observe that both normal traffic and attacks reach their maximum around 0.5–0.6 on the logarithmic axis. This means that, unlike the *count* feature in the KDD dataset, for this particular feature there is no clear boundary between attack and normal traffic. This leads to the conclusion that some parameters may serve as strong standalone anomaly predictors, while others exhibit significant overlap in value ranges for normal traffic and attacks. This highlights the necessity of using multidimensional models such as the *Autoencoder*, which, instead of relying on single indicators, learn complex nonlinear correlations among multiple features simultaneously, enabling effective classification even in the presence of ambiguity in individual attributes.

V. Results

A. Analysis of coefficient changes depending on the selected decision threshold

Table **??** presents the results of the model evaluation on the KDD dataset. The row corresponding to the percentile (p) for which the model achieved the highest *F1-Score* is highlighted in bold. The *FP* column shows the number of false alarms (*False Positives*), while the *TP* column presents the number of correctly detected attacks (*True Positives*) (values averaged over 5 runs).

Figure **??** presents the results for the first dataset (KDD99). The plot illustrates the relationship between *Recall*, *Precision*, and *F1-Score* depending on the selected percentile (decision threshold). It can be observed that for a low threshold (percentile = 60), the model achieves perfect recall (detects all attacks), but at the cost of low precision, resulting in a large

number of false alarms. As the threshold increases, recall decreases while precision improves. Consequently, the number of false alarms drops, but not all attacks are detected. The best F1 value on this dataset was achieved at the 98th percentile.

Table **??** presents the results of the model evaluation on the NetFlow V9 dataset. The row corresponding to the percentile (p) for which the model achieved the highest *F1-Score* is highlighted in bold. The *FP* column shows the number of *False Positives*, and the *TP* column shows the number of detected attacks (*True Positives*) (averaged over 5 runs).

Figure **??** presents the results for the second dataset (NetFlow V9). Similarly to the KDD99 dataset, for low percentile values the model detects nearly 100% of attacks, but this comes at the cost of a high number of false alarms. The highest *F1-Score* was achieved at the 95th percentile. As the percentile increases further, the model's recall begins to drop sharply. When the reconstruction error threshold is set too restrictively, the model starts to lose its ability to detect attacks.

Table **??** presents the results of the model evaluation on the cores-iot dataset. The row corresponding to the percentile (p) for which the model achieved the highest *F1-Score* is highlighted in bold. The *FP* column shows the number of *False Positives*, and the *TP* column shows the number of detected attacks (*True Positives*) (averaged over 5 runs).

Figure **??** presents the results for the third dataset (cores-iot). The plot shows the relationship between *Recall*, *Precision*, and *F1-Score* depending on the selected percentile (decision threshold). Similarly to the NetFlow V9 dataset, beyond a certain point (here at the 80th percentile), the model rapidly loses its ability to effectively detect network attacks. For lower percentiles (higher reconstruction error threshold), the model is able to detect over 99% of attacks in this dataset.

B. Reconstruction Error Histograms

Figure **??** shows the distribution of reconstruction errors for normal traffic and attacks (anomalies) in the KDD99 dataset. It can be observed that both normal traffic and most attacks generate reconstruction errors in the range 0.0–2.0, but the red curve (attacks) is slightly shifted to the right compared to the green curve (normal traffic). Therefore, effective separation of normal traffic and attacks is possible for this dataset, but it requires setting a very low reconstruction error threshold (0.12). This allows detection of the vast majority of attacks with a relatively small number of false alarms (Table **??**).

Figure **??** shows the reconstruction error distribution for normal traffic and attacks in the NetFlow V9 dataset. This dataset is more challenging for the implemented *Autoencoder*, as the group of attacks exhibits low reconstruction error values (below 2.0), while a significant portion of normal traffic reaches reconstruction errors close to 20.0. Thus, attacks turned out to be very similar to the portion of normal traffic best learned by the model, while within normal traffic there exists a subgroup of packets that are very complex or rare. Nevertheless, with an appropriate threshold the model is able to detect over 90% of attacks, albeit at the cost of a higher number of false alarms (Table **??**).

Figure **??** shows the reconstruction error distribution for normal traffic and attacks in the cores-iot dataset. In this dataset, a substantial portion of attacks exhibits clearly higher reconstruction error than normal traffic. However, a large fraction of attacks has reconstruction error very close to that of normal traffic. Thanks to a very low error threshold (0.000019), the *Autoencoder* is able to detect over 98% of attacks in this dataset with a minimal number of false alarms (Table **??**).

C. ROC Curves

The ROC curve shown in Figure **??** for the KDD99 dataset is nearly ideal, achieving an AUC value of 0.9956. This indicates a very high ability of the model to distinguish between normal traffic and attacks. Even at a very low false positive rate (FPR), the model achieves maximum detection performance (TPR = 1.0).

The ROC curve shown in Figure **??** for the NetFlow V9 dataset achieves an AUC value of 0.9387. This indicates high discriminative capability, although the result is worse than for the KDD99 dataset. It can also be observed that detecting 100% of attacks on this dataset requires accepting a higher number of false alarms.

The ROC curve shown in Figure **??** for the cores-iot dataset achieves an AUC value of 0.8956. Thus, for this dataset the model also achieved high effectiveness in distinguishing attacks from normal network traffic, although the result is lower than for the two previous datasets.

VI. Comparison with Other Models

Figure **??** presents a comparison of the *Recall* values between the *Autoencoder*, *Isolation Forest*, and *OC SVM* models for the KDD dataset depending on the selected decision threshold. The *Autoencoder* and *Isolation Forest* exhibit very similar, high performance. Both models maintain Recall close to 100% up to the 95th percentile, meaning they are able to detect nearly all attacks even under relatively strict thresholds. Only at very high percentiles (98–99) does recall begin to slightly decrease. Interestingly, *Isolation Forest* performs slightly better at the extreme end, maintaining higher recall. In contrast, the *OC SVM* model exhibits critical instability above the 90th percentile, where Recall drops to around 40%, and at the 99th percentile it approaches zero. This indicates that the decision function of this model correlates poorly with the actual nature of attacks in the extreme value region.

Figure **??** presents a comparison of *Precision* values between the *Autoencoder*, *Isolation Forest*, and *OC SVM* models for the KDD dataset. The *Autoencoder* and *Isolation Forest* show almost identical behavior, similar to the Recall analysis. Both models exhibit a steadily increasing characteristic, reaching values close to 1.0 at the 99th percentile. This means that as the threshold becomes more restrictive, the probability that a detected anomaly is a true attack increases significantly. In contrast, the *OC SVM* achieves results comparable to the other two models up to the 90th percentile, after which a dramatic collapse occurs. This suggests that among the samples classified by *OC SVM* as the "most extreme anomalies" (top 5% of decision function scores), a large portion actually corresponds to normal traffic.

Figure **??** presents a comparison of the *F1-Score* between the *Autoencoder*, *Isolation Forest*, and *OC SVM* models for the KDD dataset. It confirms the trend observed in Figures **??** and **??**. The *Autoencoder* and *Isolation Forest* achieve nearly identical values across most of the range. The F1-Score increases as the threshold becomes stricter from the 60th to the 98th percentile. This is because at lower thresholds the models generate many false alarms (low precision), which reduces the F1 value. Increasing the threshold significantly improves precision while maintaining high recall. Both models achieve a maximum F1 above 0.95, demonstrating very high effectiveness on this dataset. It is also worth noting that, unlike the *Autoencoder*, the *Isolation Forest* does not exhibit a drop at the 99th percentile but instead maintains or even slightly improves its performance. This suggests that the anomaly ranking produced by *Isolation Forest* is slightly "cleaner" at the extreme end, whereas the *Autoencoder* still contains some difficult-to-distinguish normal traffic or misses some attacks. The plot also highlights the weakness of the *OC SVM* model. Above the 90th percentile, the F1-Score drops sharply from around 0.85 to approximately 0.10. This means that the model is unable to operate effectively in a high-precision regime – attempts to minimize false alarms result in a complete loss of attack detection capability.

VII. Conclusions

Regarding the first research question (*Q1*), the conducted study confirmed that the *Autoencoder* architecture is an effective tool for anomaly detection in network traffic. The achieved ROC AUC values (ranging between 0.8977 and 0.9975 depending on the dataset) indicate a high capability of the model to generalize and distinguish normal traffic from attacks. The model successfully learned to compress and reconstruct the characteristics of normal traffic, treating attacks as anomalies.

A key element of success was the selection of an appropriate data preprocessing method. Raw network data exhibit power-law distributions, which impede the convergence of neural networks. Applying a logarithmic transformation combined with *RobustScaler* enabled effective class separation.

The study revealed a significant trade-off between system recall and precision. Reconstruction error density plots demonstrated that, depending on the dataset, distinguishing attacks from normal network traffic can be challenging for the model. The adopted strategy for setting the decision threshold prioritized high attack detection rates (maximizing recall), which is desirable in cybersecurity. However, this came at the cost of an increased number of false positives in areas where complex normal traffic mathematically resembles anomalies.

Comparative analysis showed that the proposed *Autoencoder* achieves performance similar to the *Isolation Forest* algorithm, maintaining a high F1-score. In contrast, the *One-Class SVM* model exhibited significant instability and a sharp drop in recall under strict settings, whereas the neural network maintained detection effectiveness even under very high precision requirements. This confirms that the reconstruction-based approach provides a competitive and stable alternative to classical machine learning methods.

The answer to the second research question (*Q2*) is affirmative. The applied *open-set recognition* training strategy (training exclusively on normal data) allowed the model to treat any type of attack—whether known during training or a novel Zero-Day threat—as an anomaly.

The project demonstrates that *Autoencoders* can serve as a solid foundation for modern network attack detection systems. However, in the case of sophisticated threats that closely mimic normal traffic patterns, the sole use of this architecture may be insufficient and may require support from additional verification mechanisms.

A. Declaration of AI Tool Usage

1) Google Gemini:

- **Usage:** Large Language Model (LLM)
- **Scope:** Assistance in writing scripts for data loading and processing, generating plots, and editing and proofreading text.
- **Notes:** All AI-generated content was verified and finalized by the authors of the work.

References

[1] https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html
[2] https://www.kaggle.com/datasets/ashtcoder/network-data-schema-in-the-netflow-v9-format
[3] https://docs.python.org/3/
[4] https://scikit-learn.org/stable/
[5] https://docs.pytorch.org/docs/stable/index.html
[6] https://pandas.pydata.org/docs/
[7] https://numpy.org/doc/2.3/
[8] https://matplotlib.org/stable/index.html
[9] https://colab.research.google.com/
[10] Mahmoud Said Elsayed, Nhien-An Le-Khac, Soumyabrata Dev, and Anca Delia Jurcut. 2020. Network Anomaly Detection Using LSTM Based Autoencoder. In Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks (Q2SWinet '20). Association for Computing Machinery, New York, NY, USA, 37–45. https://doi.org/10.1145/3416013.3426457
[11] Geng, Chuanxing, Sheng-jun Huang, and Songcan Chen. "Recent advances in open set recognition: A survey." IEEE transactions on pattern analysis and machine intelligence 43.10 (2020): 3614-3631.
[12] KDD Cup 1999 Data, https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html
[13] Network data schema in the Netflow V9 format, https://www.kaggle.com/datasets/ashtcoder/network-data-schema-in-the-netflow-v9-format
[14] Lesouple, J., Baudoin, C., Spigai, M., & Tourneret, J. Y. (2021). Generalized isolation forest for anomaly detection. Pattern Recognition Letters, 149, 109-119.
[15] Hejazi, Maryamsadat, and Yashwant Prasad Singh. "One-class support vector machines approach to anomaly detection." Applied Artificial Intelligence 27.5 (2013): 351-366.
[16] Shieh, Chin-Shiuh, et al. "Detection of unknown DDoS attack using reconstruct error and one-class SVM featuring stochastic gradient descent." Mathematics 11.1 (2022): 108.
[17] Anguita, Davide, et al. "The'K'in K-fold Cross Validation." Esann. Vol. 102. 2012.